

Kotlin では、`null` 値を指定することが可能です。Kotlinでは、何かが欠けている場合やまだ設定されていない場合に、`null`値が使用されます。コレクションの章で、マップに存在しないキーを持つキーと値のペアにアクセスしようとした際、Kotlinが`null`値を返す例をすでにご覧になったはずです。このように `null` 値を使用するのは便利ですが、コードが `null` 値の処理に対応していない場合、問題が発生する可能性があります。

プログラムにおける `null` 値に関連する問題を未然に防ぐため、Kotlin には null 安全機能が備わっています。null 安全機能は、実行時ではなくコンパイル時に `null` 値に関連する潜在的な問題を検出します。

ヌルセーフティとは、以下のことが可能になる一連の機能のことです：

- プログラム内で `null` 値が許容される場合は、それを明示的に宣言してください。
- `null` 値がないか確認してください。
- `null` 値を含む可能性があるプロパティや関数への呼び出しでは、安全な呼び出しを使用してください。
- `null` 値が検出された場合の対処方法を明記する。

Nullable types

Kotlin はヌル許容型をサポートしており、宣言された型が `null` 値を持つことが可能になります。デフォルトでは、型が `null` 値を受け入れることは許可されていません。Null許容型は、型宣言の後に明示的に `?` を追加することで宣言されます。

例えば：

```
fun main() {
    // neverNull has String type
    var neverNull: String = "This can't be null"

    // Throws a compiler error
    neverNull = null

    // nullable has nullable String type
    var nullable: String? = "You can keep a null here"

    // This is OK
    nullable = null

    // デフォルトでは、null値は受け入れられません
    var inferredNonNull = "The compiler assumes non-nullable"

    // Throws a compiler error
    inferredNonNull = null

    // notNull doesn't accept null values
    fun strLength(notNull: String): Int {
        return notNull.length
    }

    println(strLength(neverNull)) // 18
}
```

```
println(strLength(nullable)) // Throws a compiler error
}
```

{kotlin-runnable="true" validate="false" kotlin-min-compiler-version="1.3" id="kotlin-tour-nullable-type"}

`length` は、文字列に含まれる文字数を表す `String` クラスのプロパティです。

{style="tip"}

Check for null values

ヌル値の有無を確認する

条件式の中で `null` 値が含まれているかどうかを確認することができます。次の例では、`describeString()` 関数には、`maybeString` が `null` ではないかどうか、およびその `length` が 0 より大きいかどうかを確認する `if` 文が含まれています。

```
fun describeString(maybeString: String?): String {
    if (maybeString != null && maybeString.length > 0) {
        return "String of length ${maybeString.length}"
    } else {
        return "Empty or null string"
    }
}

fun main() {
    val nullString: String? = null
    println(describeString(nullString))
    // Empty or null string
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-check-nulls"}

Use safe calls

安全な呼び出しの使用

`null` 値を含む可能性があるオブジェクトのプロパティに安全にアクセスするには、セーフコール演算子 `?.` を使用します。safe call 演算子は、オブジェクトまたはそのアクセスされたプロパティのいずれかが `null` の場合、`null` を返します。これは、コード内で `null` 値が存在することでエラーが発生するのを避けたい場合に役立ちます。

次の例では、`lengthString()` 関数はセーフコールを使用して、文字列の長さか `null` のいずれかを返します：

```
fun lengthString(maybeString: String?): Int? = maybeString?.length

fun main() {
```

```
val nullString: String? = null
println(lengthString(nullString))
// null
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-safe-call-property"}

セーフコールは連鎖させることができるため、オブジェクトのいずれかのプロパティが `null` 値を含む場合、エラーをスローすることなく `null` が返されます。例えば：

```
person.company?.address?.country
```

{style="tip"}

safe call 演算子は、拡張関数やメンバ関数を安全に呼び出すためにも使用できます。この場合、関数が呼び出される前にヌルチェックが行われます。チェックの結果、`null` 値が検出された場合、その呼び出しはスキップされ、`null` が返されます。

次の例では、`nullString` は `null` であるため、`.uppercase()` の呼び出しはスキップされ、`null` が返されます：

```
fun main() {
    val nullString: String? = null
    println(nullString?.uppercase())
    // null
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-safe-call-function"}

Use Elvis operator

Elvis演算子の使用

Elvis演算子 `?:` を使用すると、`null` 値が検出された場合に返すデフォルト値を指定できます。

Elvis演算子の左側に、`null` 値かどうかを確認すべきものを記述してください。Elvis演算子の右側に、`null` 値が検出された場合に何を返すかを記述してください。

次の例では、`nullString` は `null` であるため、`length` プロパティにアクセスするためのセーフコールは `null` 値を返します。その結果、Elvis演算子は `0` を返します：

```
fun main() {  
    val nullString: String? = null  
    println(nullString?.length ?: 0)  
    // 0  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-elvis-operator"}

Kotlin のヌルセーフティに関する詳細については、[Null safety](#) を参照してください。

演習 {completion-point="true"}

課題 {initial-collapse-state="collapsed" collapsible="true"}

`employeeById` という関数があり、これを使えば、ある会社の従業員データベースにアクセスできます。残念ながら、この関数は `Employee?` 型の値を返すため、結果が `null` になる可能性があります。この課題の目的は、従業員の `id` が指定された場合にその従業員の給与を返し、データベースにその従業員が存在しない場合は `0` を返す関数を作成することです。

|---|---|

```
data class Employee (val name: String, var salary: Int)  
  
fun employeeById(id: Int) = when(id) {  
    1 -> Employee("Mary", 20)  
    2 -> null  
    3 -> Employee("John", 21)  
    4 -> Employee("Ann", 23)  
    else -> null  
}  
  
fun salaryById(id: Int) = // Write your code here  
  
fun main() {  
    println((1..5).sumOf { id -> salaryById(id) })  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-null-safety-exercise"}

|---|---|

```
data class Employee (val name: String, var salary: Int)  
  
fun employeeById(id: Int) = when(id) {  
    1 -> Employee("Mary", 20)  
    2 -> null  
    3 -> Employee("John", 21)  
}
```

```
4 -> Employee("Ann", 23)
else -> null
}

fun salaryById(id: Int) = employeeById(id)?.salary ?: 0

fun main() {
    println((1..5).sumOf { id -> salaryById(id) })
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-null-safety-solution"}

What's next?

次は何？

おめでとうございます！初心者向けツアーを完了しましたので、次は中級者向けツアーでKotlinの理解をさらに深めましょう：

- [Previous step](#)
- [Start intermediate Kotlin tour](#)